



*Seguridad en el Desarrollo de
Aplicaciones*

PRÁCTICA: AGREGAR TAREA EN EL KANBAN

Valeria Hernández Cortés

Grupo IDGS17

Ediciones del archivo kanban.component.html

1. Se creó el botón "Crear nueva tarea", el cual permite al usuario abrir un cuadro de diálogo para registrar una nueva tarea.

```
15 <div style="text-align: center; margin-bottom: 30px;">
16   <button mat-raised-button class="btn-snoopy-create" (click)="openCreateDialog()">
17     <mat-icon>add</mat-icon> Crear nueva tarea "Crear": Unknown word.
18   </button>
19 </div>
```

2. Se implementó un cuadro de diálogo que contiene los campos necesarios para registrar una nueva tarea. Los valores para crear la nueva tarea se almacenan en el objeto newTask. Los campos Título de la tarea y Descripción son obligatorios mediante required. Además, se agregaron mensajes de validación que se muestran cuando el usuario deja un campo vacío.

```
<ng-template #createTaskDialog>
  <h2 mat-dialog-title class="dialog-title">Añadir Tarea</h2> "Añadir": Unknown word.
  <p></p>

  <mat-dialog-content>
    <mat-form-field appearance="outline" class="dialog-field">
      <mat-label>Título de la tarea</mat-label> "Título": Unknown word.
      <input matInput [(ngModel)]="newTask.title" name="title" required #titleCtrl="ngModel" placeholder="Ej. Darle de cenar a Snoopy"> "Darle": Unknown word.

      @if (titleCtrl.invalid && (titleCtrl.dirty || titleCtrl.touched)) {
        <mat-error>El título es <strong>obligatorio</strong></mat-error> "título": Unknown word.
      }
    </mat-form-field>

    <mat-form-field appearance="outline" class="dialog-field">
      <mat-label>Descripción</mat-label> "Descripción": Unknown word.
      <textarea matInput [(ngModel)]="newTask.description" name="description" required #descCtrl="ngModel" placeholder="Ej. Comprar croquetas y servir a las 8PM..."></textarea> "Comprar": Unknown word.

      @if (descCtrl.invalid && (descCtrl.dirty || descCtrl.touched)) {
        <mat-error>La descripción es <strong>obligatoria</strong></mat-error> "descripción": Unknown word.
      }
    </mat-form-field>
  </mat-dialog-content>
</ng-template>
```

3. Se creó el botón "Agregar", su función es guardar la nueva tarea mediante la ejecución del método addTask(), se encuentra desactivado si el campo de título está inactivo.

```
<button mat-raised-button (click)="addTask()" [disabled]="titleCtrl.invalid || descCtrl.invalid"
  class="btn-snoopy-save">Agregar</button> "Agregar": Unknown word.
```

4. Se implementó de igual manera el botón "Cancelar" que permite cerrar el cuadro de diálogo sin guardar información; al presionarlo se ejecuta el método closeDialog()

```
<mat-dialog-actions align="end" class="dialog-actions">
  <button mat-button (click)="closeDialog()" class="btn-snoopy-cancel">Cancelar</button>
```

Ediciones del archivo kanban.component.html

1. Se declararon las variables necesarias para almacenar los datos de la nueva tarea (newTask), controlar el cuadro de diálogo (createTaskDialog) y mostrar mensajes de confirmación (successMessage). También se integró el servicio MatSnackBar para mostrar notificaciones al usuario.

```
34 export class KanbanComponent implements OnInit {
35   @ViewChild('createTaskDialog') createTaskDialog!: TemplateRef<any>;
36
37   newTask = { title: '', description: '' };
38   todo: TaskDto[] = [];
39   done: TaskDto[] = [];
40   successMessage: string | null = null;
41
42   constructor(
43     private taskService: TaskService,
44     private cdr: ChangeDetectorRef,
45     private dialog: MatDialog,
46     private snackBar: MatSnackBar
47   ) {}
48 }
```

2. Se implementaron los métodos para abrir y cerrar el cuadro de diálogo. Cada vez que se abre el formulario, los campos se reinician para que el usuario pueda capturar una nueva tarea desde cero.

```
69   openCreateDialog() {
70     this.newTask = { title: '', description: '' };
71     this.dialog.open(this.createTaskDialog, {
72       width: '400px',
73     });
74   }
75
76   closeDialog() {
77     this.dialog.closeAll();
78   }
79 }
```

3. Se creó el método addTask(), encargado de enviar la información de la tarea al servicio mediante una petición POST. Si el registro se realiza correctamente, la nueva tarea se agrega a la lista y se muestra una notificación de éxito durante unos segundos.

```

80     addTask() {
81         if (!this.newTask.title.trim()) return;
82
83         const request: CreateTaskRequest = {
84             title: this.newTask.title,
85             description: this.newTask.description
86         };
87
88         this.taskService.createTask(request).subscribe({
89             next: (task: TaskDto) => {
90                 this.todo.unshift(task);
91                 this.closeDialog();
92
93                 this.successMessage = 'Tarea creada con éxito.';
94                 this.cdr.detectChanges();
95
96                 setTimeout(() => {
97                     this.successMessage = null;
98                     this.cdr.detectChanges();
99                 }, 3000);
100             },

```

4. En caso de que ocurra algún error se utiliza MatSnackBar para mostrar un mensaje informativo en la parte inferior de la pantalla que permite al usuario saber que la operación no pudo completarse exitosamente.

```

101         error: () => {
102             this.snackBar.open('Hubo un error al crear la tarea. Intenta de nuevo.', 'Cerrar', {
103                 duration: 5000,
104                 panelClass: ['error-snackbar']
105             });
106         }
107     });
108 }

```

Pruebas de funcionamiento

Botón de Creación



Apertura del Cuadro de Diálogo



Validación de campos



GESTIÓN DE TAREAS



AÑADIR TAREA

Título de la tarea*

Titulo

Descripción*

Ej. Comprar croquetas y servir a las 8PM...

//

La descripción es **obligatoria**

Cancelar

Agregar

Confirmación de Éxito



GESTIÓN DE TAREAS



Tarea creada con éxito.

+ Crear nueva tarea



PENDIENTES

Tarea 07

Tarea nueva

Snoppy



COMPLETADAS

Tarea 01

Comprar la despensa

Tarea 05

“Defiende tus decisiones”

1. ¿Por qué elegiste esta forma de manejar el formulario?

Porque es la forma que he utilizado los formularios desde siempre. Me permite capturar los datos fácilmente, validar que los campos estén llenos y enviar la información sin problemas.

2. ¿Qué pasa si el usuario hace clic en "Guardar" dos veces seguido?

No pasa nada, porque cuando la tarea se guarda correctamente el cuadro de diálogo se cierra de inmediato. Ya no hay oportunidad de volver a presionar el botón.

3. ¿Cómo sabe tu aplicación que la tarea se creó correctamente?

Porque el backend envía una respuesta de éxito. Cuando la aplicación recibe esa respuesta, agrega la tarea a la lista y muestra un mensaje de confirmación.

4. Si el backend falla, ¿qué ve el usuario?

Ve un mensaje indicando que ocurrió un error al guardar la tarea. Así sabe que la operación no se completó y puede intentarlo nuevamente.

5. ¿Por qué reseteaste el formulario después de guardar?

Para que cuando se abra otra vez esté vacío y listo para capturar una nueva tarea. Así se evita que queden datos de la tarea anterior.

Declaración de uso de IA

Durante el desarrollo del módulo de tareas utilicé Inteligencia Artificial (Gemini) como apoyo técnico. Me ayudó a:

1. Mejorar partes del código
2. Resolver dudas sobre Angular Material
3. Entender errores de conexión entre el frontend y el backend
4. Verificar que las funcionalidades estuvieran implementadas correctamente
5. Organizar y redactar la documentación final